

```
for health1 in range(healthvalue):
    screen.blit(health, (health1+8,8))
```

The code first draws the all-red health bar. Then, it draws a certain amount of green over the bar, according to how much life the castle has remaining. If you build and run the program, you would have a timer and a health bar.

Win or lose. You need some sort of a win or lose scenario to make the game worth playing. Do this by exiting out of the main loop and going into a win or lose loop. In this loop, figure out if the user won or lost, and display that on the screen accordingly.

Add the following lines to the end of game.py:

```
#10 - Win/Lose check
if pygame.time.get_ticks() >= 90000:
    running=0
    exitcode=1
if healthvalue<0:
    running=0
    exitcode=0
if acc[1]!=0:
    accuracy=acc[0]*1.0/acc[1]*100
else:
    accuracy=0
# 11 - Win/Lose display
if exitcode==0:
    pygame.font.init()
    font = pygame.font.Font(None, 24)
    text = font.render("Accuracy: "+str(accuracy)+"%", True, (255,0,0))
    textRect = text.get_rect()
    textRect.centerx = screen.get_rect().centerx
    textRect.centery = screen.get_rect().centery+24
    screen.blit(gameover, (0,0))
    screen.blit(text, textRect)
else:
    pygame.font.init()
    font = pygame.font.Font(None, 24)
    pygame.display.flip()
```

The first 'if' statement checks if the time is up. The second checks if the castle is destroyed. The third calculates the accuracy ratio. After that, a quick 'if' statement checks if you won or lost, and displays the correct image.

Note: The `acc[0]*1.0` is just

converting `acc[0]` to a float. If you do not do this, the division operand will return an integer like 1 or 2 instead of a float like 1.5.

If you want to display images for the win and lose screens, then those images have to be loaded first. To do so, add the following code to the end of section #3:

```
gameover = pygame.image.load("resources/
images/gameover.png")
youwin = pygame.image.load("resources/
images/youwin.png")
```

Change section #4 to:

```
# 4 - keep looping through
```

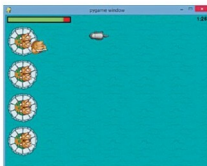


Fig. 5: A mole on the screen

```
running = 1
exitcode = 0
while running:
    badtimer-=1
```

The running variable checks if the game is over and the exit code variable tells whether the player won or lost.

Gratuitous music and sound effects. PyGame makes loading and playing sounds simple. First, initialise the mixer by adding the following at the end of section #2:

```
pygame.mixer.init()
Then, load the sounds and set the volume level at the end of section #3.
# 3.1 - Load audio
hit = pygame.mixer.Sound("resources/
audio/explode.wav")
```

```
enemy = pygame.mixer.Sound("resources/
audio/enemy.wav")
shoot = pygame.mixer.Sound("resources/
audio/shoot.wav")
hit.set_volume(0.05)
enemy.set_volume(0.05)
```

Most of the above code is simply loading audio files and configuring audio volume levels. But, pay attention to `pygame.mixer.music.load` line. It loads background music for the game and the next line sets the background music to repeat forever.

This takes care of the audio configuration. Play various sound effects as needed. Do that as per the directions in the comments for the code given below:

```
# section 6.3.1 after if badrect.
left<64:
    hit.play()
# section 6.3.2 after if badrect.
colliderect(bullrect):
    enemy.play()
# section 8, after if event.
type==pygame.MOUSEBUTTONDOWN:
    shoot.play()
```

You now have background music and sound effects for collisions and shooting, and the game is much more alive.

Where to go from here. You have just finished creating a fun game packed with music, sound effects, a killer bunny, kamikaze moles and more. At this point, feel free to modify this game. May be, replace the art with your own drawings, or add different weapons or types of monsters into the game.

Testing the game

Before running the game, install library file `pygame-1.9.1.win32-py2.7`. Make sure the resource files and `game.py` are in same folder. Then, click on `game.py` file and the game will automatically start. Use your mouse to shoot the moles. **EFY**

EFY Note

1. The source code of this project is included in this month's EFY DVD and is also available for free download at source.efymag.com

2. The game was tested on Windows 7 at EFY Lab



Rajasekar Mutukuri is pursuing BE at S.R.K.R Engineering College, Bhimavaram